

Vel Tech Group of Institutions

Name: Gontu Yaswanth

Email: vtu33434@veltech.edu.in

Roll no: Vtu33434

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Competitive Coding II_L8

VTU_Week 2_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Neha is organizing a sequence of integers where some elements are zeroes. She wants to rearrange the sequence so that all zeroes are moved to the end while keeping the relative order of the non-zero elements intact. Help Neha perform this task efficiently using the two-pointer technique to modify the sequence in place.

Input Format

The first line of input consists of an integer n , representing the size of the array `nums[]`.

The second line contains n integers, representing the elements of the array `nums[]`.

Output Format

The output prints the modified array `nums[]`, with all zeroes moved to the end and the relative order of the non-zero elements maintained.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

0 1 0 3 12

Output: 1 3 12 0 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
void moveZeroes(int n, int nums[]) {  
    int pos = 0;  
  
    for (int i = 0; i < n; i++) {  
        // If the current element is non-zero  
        if (nums[i] != 0) {  
            // Swap the current element with the element at 'pos'  
            int temp = nums[i];  
            nums[i] = nums[pos];  
            nums[pos] = temp;  
  
            // Move the 'pos' pointer to the next available slot  
            pos++;  
        }  
    }  
}
```

```
int main() {  
    int n;  
    if (scanf("%d", &n) != 1) return 0;  
  
    int nums[100]; // Based on constraints n <= 100  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &nums[i]);  
    }  
}
```

```

moveZeroes(n, nums);

// Print the modified array
for (int i = 0; i < n; i++) {
    printf("%d", nums[i]);
    if (i < n - 1) {
        printf(" ");
    }
}
printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Arjun has a list of integers representing a specific sequence, and he wants to find its next permutation in lexicographical order. If the sequence is the largest permutation possible, he should rearrange it into the smallest permutation. Help Arjun solve this problem efficiently using the two-pointer technique to rearrange the sequence in place with constant extra memory.

Input Format

The first line of input consists of an integer n , representing the size of the array `nums[]`.

The second line contains n integers, representing the elements of the array `nums[]`.

Output Format

The output prints "Next permutation:" followed by n space-separated integers on the next line, representing the next lexicographically greater permutation of the given sequence.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

Output: Next permutation:

1 3 2

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
// Helper function to swap two elements
```

```
void swap(int *a, int *b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
// Helper function to reverse a portion of the array
```

```
void reverse(int nums[], int start, int end) {
```

```
    while (start < end) {
```

```
        swap(&nums[start], &nums[end]);
```

```
        start++;
```

```
        end--;
```

```
    }
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    if (scanf("%d", &n) != 1) return 0;
```

```
    int nums[10]; // Based on constraints n <= 10
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &nums[i]);
```

```
    }
```

```
    int i = n - 2;
```

```
    // Step 1: Find the first decreasing element from the right
```

```
    while (i >= 0 && nums[i] >= nums[i + 1]) {
```

```
        i--;
```

```
    }
```

```

if (i >= 0) {
    // Step 2: Find the element just larger than nums[i] to swap
    int j = n - 1;
    while (nums[j] <= nums[i]) {
        j--;
    }
    swap(&nums[i], &nums[j]);
}

// Step 3: Reverse the elements to the right of the pivot
reverse(nums, i + 1, n - 1);

// Output formatting
printf("Next permutation:\n");
for (int k = 0; k < n; k++) {
    printf("%d%s", nums[k], (k == n - 1 ? "" : " "));
}
printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Priya is searching for a specific word (needle) in a large text (haystack). She needs to find the index of the first occurrence of the word in the text. If the word is not present, she should return -1. Help Priya solve this efficiently using the two-pointer technique.

Input Format

The first line of input consists of a string haystack, representing the text in which the word is to be searched.

The second line of input consists of a string needle, representing the word to search for in the text.

Output Format

The output prints "The index of the first occurrence of needle in haystack is: X", where X represents the index value.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: sadbutsad
sad

Output: The index of the first occurrence of needle in haystack is: 0

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

int main() {
    char haystack[101];
    char needle[101];

    // Read the strings. %s stops at whitespace/newline.
    if (scanf("%s", haystack) != 1) return 0;
    if (scanf("%s", needle) != 1) return 0;

    int hLen = strlen(haystack);
    int nLen = strlen(needle);
    int result = -1;

    // We only need to slide the window up to the point where
    // the remaining haystack length is at least the needle length.
    for (int i = 0; i <= hLen - nLen; i++) {
        int j = 0;

        // Compare needle with the current segment of haystack
        while (j < nLen && haystack[i + j] == needle[j]) {
            j++;
        }

        // If the inner pointer reached the end of the needle, we found a match
        if (j == nLen) {
```

```

        result = i;
        break; // Stop at the first occurrence
    }
}

// Strict formatting: check colons, spaces, and case sensitivity
printf("The index of the first occurrence of needle in haystack is: %d", result);

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Ramesh is working with an array of integers and needs to remove all occurrences of a specific value from it.

To accomplish this efficiently, he uses the Two-Pointer technique, where one pointer scans the array and another pointer tracks the position to place valid elements. Given an integer array `nums` and an integer `val`, Ramesh must modify the array in-place so that all elements equal to `val` are removed while preserving the order of the remaining elements.

Write a program to ensure that the number of remaining elements and the modified array are printed correctly.

Input Format

The first line contains an integer `n`, representing the number of elements in the array.

The second line contains `n` space-separated integers, representing the elements of the array.

The third line contains an integer `val`, representing the value to be removed from the array.

Output Format

The first line prints "Number of elements after removal: " followed by count of elements not equal to `val`.

The second line prints "Modified array:"

Then print the modified array elements (up to index k) separated by a single space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3 2 2 3

3

Output: Number of elements after removal: 2

Modified array:

2 2

Answer

// You are using GCC

#include <stdio.h>

```
int main() {  
    int n, val;
```

```
    // Read array size  
    if (scanf("%d", &n) != 1) return 0;
```

```
    int nums[15]; // Max constraint is 15  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &nums[i]);  
    }
```

```
    // Read value to remove  
    if (scanf("%d", &val) != 1) return 0;
```

```
    int k = 0; // Tracks position for elements not equal to val  
    for (int i = 0; i < n; i++) {  
        if (nums[i] != val) {  
            nums[k] = nums[i];  
            k++;  
        }
```



```

    }
}

// Output formatting strictly following the rules
printf("Number of elements after removal: %d\n", k);
printf("Modified array:\n");
for (int i = 0; i < k; i++) {
    printf("%d%s", nums[i], (i == k - 1 ? "" : " "));
}
printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Anil is working with a sorted array of integers and wants to remove duplicate elements efficiently. To solve this problem, he uses the Two-Pointer technique, where one pointer scans the array and another pointer keeps track of the position to store unique elements. Given a sorted integer array `nums`, Anil must modify the array in-place so that each unique element appears only once.

Write a program to ensure that the number of unique elements and the modified array are printed correctly.

Input Format

The first line contains an integer `n`, representing the number of elements in the array.

The second line contains `n` space-separated integers, representing the elements of the sorted array.

Output Format

The first line prints "Number of unique elements: `k`"

On the next line, print "Modified array:"

Then, print the first k elements of the modified array, separated by a single space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8

-1 -1 0 0 1 1 2 2

Output: Number of unique elements: 4

Modified array:

-1 0 1 2

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    // Read the number of elements
```

```
    if (scanf("%d", &n) != 1) return 0;
```

```
    // Handle empty array case based on constraints (1 <= n)
```

```
    if (n == 0) {
```

```
        printf("Number of unique elements: 0\nModified array:\n\n");
```

```
        return 0;
```

```
    }
```

```
    int nums[15];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &nums[i]);
```

```
    }
```

```
    // Two-pointer logic
```

```
    // k tracks the index of the last unique element found
```

```
    int k = 0;
```

```
    for (int i = 1; i < n; i++) {
```

```
        // If the current element is different from the last unique element
```

```
        if (nums[i] != nums[k]) {
```

```
        k++;  
        nums[k] = nums[i];  
    }  
}  
  
// Output results  
// The number of unique elements is k + 1  
printf("Number of unique elements: %d\n", k + 1);  
printf("Modified array:\n");  
for (int i = 0; i <= k; i++) {  
    printf("%d%s", nums[i], (i == k ? "" : " "));  
}  
printf("\n");  
return 0;  
}
```

Status : Correct

Marks : 10/10